

angr-Hardness Toolchain

Mathematics, Physics & Implementation
A Complete Reference — C1 to C6 Pipeline

AUTHOR	Stuart Thomas
DATE	2026-04-17
PLATFORM	macOS arm64e · angr · numpy · scipy · claripy
STATUS	Public methodology — findings redacted pending disclosure
REPOSITORY	macos_vuln_toolchain/angr_hardness/
STAGES	C1, C2, C3, C6 implemented · C4, C5 killed by design review

Contents

§1	The Short Version — for people who don't want the maths
§2	Why C1–C6? The Naming History
§3	C1 — Phase-Transition Symbolic Execution
§4	C2 — Random Matrix Theory Call Graph Screen
§5	C3 — Template-Based Call Dataflow Matching
§6	C6 — Symbolic Taint Analysis
§7	The Disassembly Layer
§8	Pipeline Composition
§9	Limitations & Dead Ends
§10	Glossary

§1

The Short Version

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

1.1 What problem are we solving?

macOS ships with several hundred privileged daemons — programs that run as root, talk to the network, handle files, or manage security policy. Any one of them could contain a bug. Finding bugs manually means reading disassembly for weeks. Fuzzing blindly wastes compute on code hardened for years. We need a way to look at a compiled binary — no source, no symbols — and answer: which function should I look at first?

1.2 The city analogy

Every program is a city. Roads are connections between neighbourhoods (functions). Some cities are planned: regular grid, sensible layout. Others grew organically and have one enormous junction where everything passes through, weird dead ends, and roads that loop back for no obvious reason. The weird city is more likely to have a problem — not because weirdness causes bugs directly, but because weird structure correlates with code that evolved under pressure and probably has corners nobody tested. The C2 stage of this toolchain does that structural audit mathematically, at scale, in about 30 seconds per binary.

1.3 What does symbolic execution mean?

When a normal program runs, it processes actual data. When angr runs it, it processes symbols — placeholders for "whatever the attacker could send." Instead of computing $x + 3 = 7$, it tracks $x + 3 = y$ and asks: is there a value of x that makes y overflow a buffer? The problem is that programs branch constantly. A program with 100 branches has 2^{100} possible paths — more than atoms in the observable universe. C1 makes symbolic execution tractable by prioritising paths that are probably solvable, using a result from statistical physics: near a phase transition, constraint systems become maximally hard. C1 estimates how close each path is to that transition and uses that estimate as a priority signal.

1.4 Why does the pipeline produce results?

Because it combines signals that individually are noisy, but together are discriminative:

- Structure (C2): the call graph looks unusual — worth investigating
- Pattern (C3): the code does receive untrusted data → allocate memory without a bounds check in between — that pattern matches known vulnerability classes
- Confirmation (C6): symbolic execution finds a concrete input that reaches the allocator with an attacker-controlled size

None of these alone is sufficient. Together, they narrow a binary with 300 functions down to 3 worth reading carefully. A 44 KB binary analysed in 30 seconds identifies the function with cyclomatic complexity 155 — which turns out to be the entire main loop. A 365 KB DLL screened in under a minute isolates the ICMPv6 reply parser with 23 callees and a recursive self-call. That is the function you want a human looking at.

The honest version

It does not always work. angr hits memory limits on large binaries (~3.3 MB ceiling).

Compiler optimisation hides patterns the tools expect.

Some bugs are architectural — no static analysis finds them without protocol semantics.

The pipeline is a triage tool, not an oracle.

§2

Why C1–C6? The Naming History

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

2.1 The Kill Chain connection

The naming comes — with deliberate irony — from the Lockheed Martin Cyber Kill Chain framework, which models an attacker's progression in seven stages: Reconnaissance, Weaponisation, Delivery, Exploitation, Installation, Command and Control, Actions on Objectives. The C-numbering (C1 through C6) is deliberately reminiscent of military chain designations — suggesting sequential dependency and the fact that each stage can be run independently. When this toolchain was designed, "chain" was reappropriated for the defender's research methodology: a sequential pipeline where each stage feeds the next, outputs flow forward, and nothing reaches the exploitation stage (writing a PoC) without passing through the earlier screens.

2.2 Purple-teaming the architecture

The original architecture had exactly six components. A design prompt was sent independently to four LLMs (ChatGPT, Grok, DeepSeek, Gemini) — a process called purple-teaming the architecture. The four responses were synthesised to kill anything theoretically elegant but operationally useless. Two stages were killed:

Stage	Proposal	Verdict	Reason
C1	Phase-aware symbolic executi...	Keep, harden	Drop Survey Propagation; use backbone fraction
C2	RMT call graph screen	Keep, fix null	M-P wrong for power-law; use config model
C3	Matched filtering on call graphs	Redesign	Instruction-NCC rejected; call-level VEX correct
C4	TDA on CFGs	✗ Kill	Persistent homology = same signal as McCabe × 1000
C5	Compressed sensing for cove...	✗ Kill	RIP fails for binary coverage bitmaps
C6	Dataflow taint analysis	Add (new)	Unanimous highest-ROI; not in original design

2.3 Ghost stages

The numbering was preserved rather than renumbered after C4 and C5 were killed. This is why the implemented pipeline jumps from C3 directly to C6 — the ghost stages are a feature, not a bug. Anyone reading the codebase can see immediately that two ideas did not survive first contact with reality. C4 was killed because persistent homology of a CFG captures loop structure and branching depth — but those are already captured by back-edge count and cyclomatic complexity, both computed in microseconds by C2. C5 was killed because compressed sensing requires the measurement matrix to satisfy the Restricted Isometry Property — and coverage bitmaps are binary vectors with highly structured sparsity patterns that violate RIP badly. There is something satisfying about a cyber kill chain that kills two of its own stages.

§3

C1 — Phase-Transition Symbolic Execution

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

3.1 The physical picture: phase transitions

The 3-SAT satisfiability problem has a phase transition. A 3-SAT instance consists of n boolean variables and m clauses, each clause constraining exactly three variables. Define the clause-to-variable ratio $\alpha = m/n$. For small α , almost every assignment satisfies all clauses — easy. For large α , the clauses are contradictory — again easy. But at exactly:

$\alpha_c \approx 4.267$ (Mézard, Montanari, Zecchina 2002)

the problem undergoes a sharp phase transition. Below α_c : satisfiable. Above: unsatisfiable. At α_c : maximally hard. The hardness is exponential — CDCL solvers take time growing exponentially with n for instances near α_c . An instance with $n=50$ variables at the transition can take longer than the age of the universe to solve optimally. The condensation transition at $\alpha_{\text{cond}} \approx 4.15\text{--}4.27$ is subtler: above it, solutions condense into a small number of clusters with large empty regions between them, causing belief-propagation and local-search algorithms to get stuck.

3.2 The backbone — the key proxy metric

The backbone of a satisfiable 3-SAT instance is the set of variables that take the same value in every solution. As α approaches α_c from below, the backbone fraction (backbone size / n) grows toward 1.0 — more variables are frozen, the solution space shrinks, and finding any solution becomes harder. C1 uses backbone fraction as its hardness estimator, computed via assumption-based probing on Z3 (not Survey Propagation, which the Tseitin CNF encoding destroys):

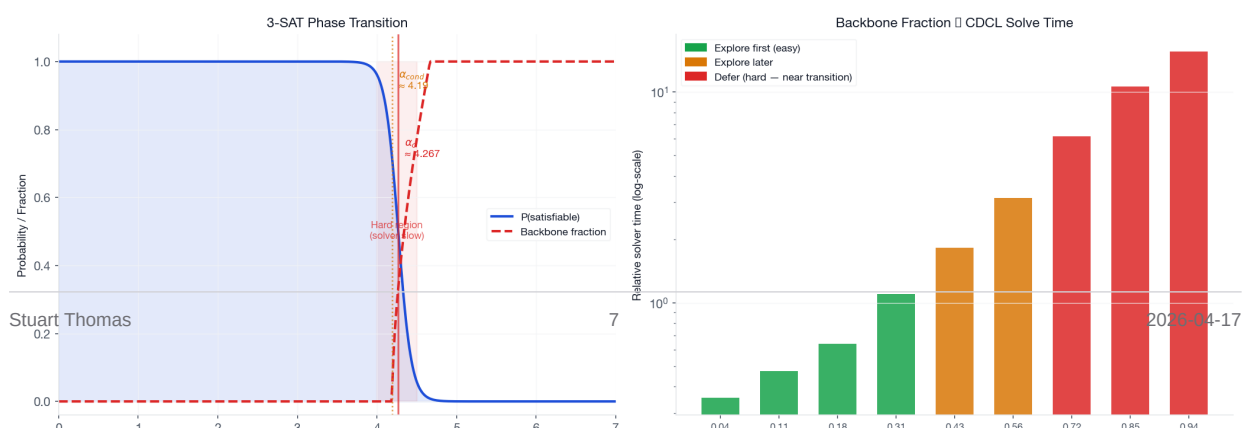
```
for var_name, width in symbolic_vars.items():
    for bit_idx in range(width):
        sat_true = solver.check(z3_var_bit == 1) == z3.sat
        sat_false = solver.check(z3_var_bit == 0) == z3.sat
        if sat_true != sat_false: # only one value works → backbone
            forced += 1
    backbone_fraction = forced / total_bits
```

Priority formula

$\text{priority}(\text{state}) = 1.0 - \text{backbone_fraction}(\text{state})$

States are sorted by priority (descending) at each exploration step. States above configurable threshold τ are moved to a `hardness_deferred` stash. The adaptive threshold mode defers the top τ -percentile of the current active stash rather than a fixed backbone cutoff, preventing runaway deferral when all paths are hard.

C1 — Phase-Transition Symbolic Execution



§4

C2 — Random Matrix Theory Screen

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

4.1 Graph theory: the adjacency matrix and its spectrum

The call graph $G = (V, E)$ has adjacency matrix A where $A[i, j] = 1$ if function i calls function j . The eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$ (solutions to $\det(A - \lambda I) = 0$) form the spectrum of G . Three statistics are computed from the spectrum:

- Spectral radius $\rho(A) = \max |\lambda_i|$ — zero for pure DAGs, positive when cycles exist; larger values indicate stronger cyclic structure
- Graph energy $E(G) = \sum |\lambda_i| / N$ — average absolute eigenvalue per node; elevated for dense, irregular graphs
- Eigenvalue entropy $H = -\sum p_i \log p_i$, $p_i = |\lambda_i| / \sum |\lambda_i|$ — low entropy signals structured hierarchy; high entropy signals non-random topology

4.2 Why NOT Marchenko-Pastur

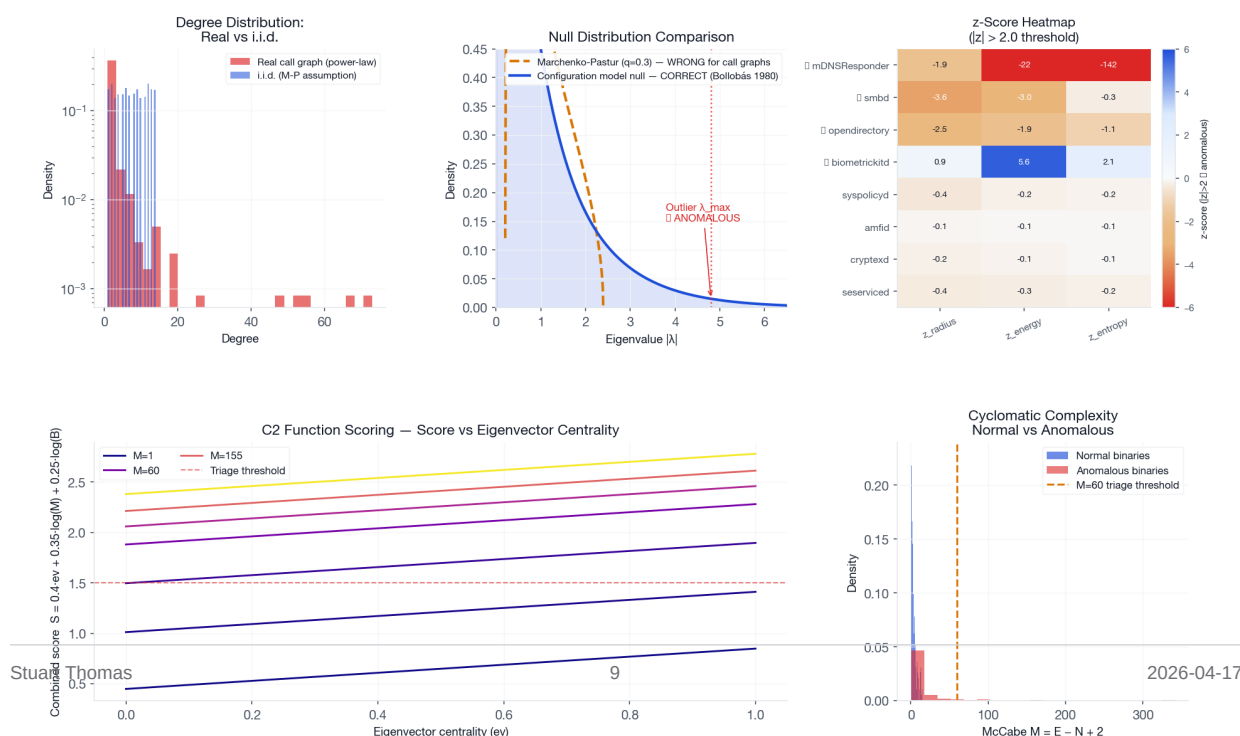
The Marchenko-Pastur distribution applies to covariance matrices of i.i.d. random matrices. Both M-P and the Wigner semicircle law assume matrix entries are i.i.d. — this fails completely for call graphs, which have power-law degree distributions (a few hub functions called by many others), bipartite-like structure, and extreme sparsity. The correct null distribution is the configuration model (Bollobás 1980, directed variant): generate a random directed graph with exactly the same in/out degree sequence as the observed graph, but random wiring. This controls for hub structure.

Z-score formula

$$z_m = (m_{\text{observed}} - \mu_{\text{null}}(m)) / \sigma_{\text{null}}(m)$$

μ_{null} and σ_{null} are estimated from 50 configuration-model replicates. Threshold: $|z| > 2.0$ flags a binary as ANOMALOUS. Both positive and negative z-scores can be anomalous: $z_{\text{energy}} = -22.33$ (mDNSResponder) indicates massively structured internal complexity; $z_{\text{entropy}} = +5.63$ (biometrickitd) indicates Swift stdlib noise confirmed as false positive.

C2 — Random Matrix Theory Screen



§5

C3 — Template-Based Call Dataflow Matching

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

5.1 Design choice: why not full SSA?

Full SSA reaching-definitions analysis would give exact dataflow information across an entire binary. It would also take hours on a binary with 45,000 functions. The key insight is that the vulnerability patterns we care about — XPC type confusion, mach_msg OOB, IOKit OOB, port UAF — all share a common structure: `source_function` produces attacker-influenced data → `sink_function` receives it. The intermediate steps may involve stack spills and loads, but the call-level structure is preserved. C3 builds a call-level def-use graph and matches templates against it — fast enough to run on all functions in seconds.

5.2 Register and stack taint tracking

C3 lifts each basic block to VEX IR and scans `WrTmp`, `Put`, and `Store` statements to track taint through registers and stack slots. The key challenge is ARM64 compiler behaviour: return values (x0) are frequently spilled to the stack between calls, then reloaded. C3 handles this via canonical address resolution:

```
GET(register)      → "register_name"      (e.g., "x0")
Add64(RdTmp(t), C) → "sp+0x10"            (frame-relative address)
Load(Add64(sp, C)) → resolves to "sp+0x10" → finds spilled taint
```

This `mem_state` dictionary keyed by canonical address strings survives cross-block spills and correctly tracks `x0` → `[sp, #0x10]` → `x0` register round-trips that would defeat a naive per-block analysis.

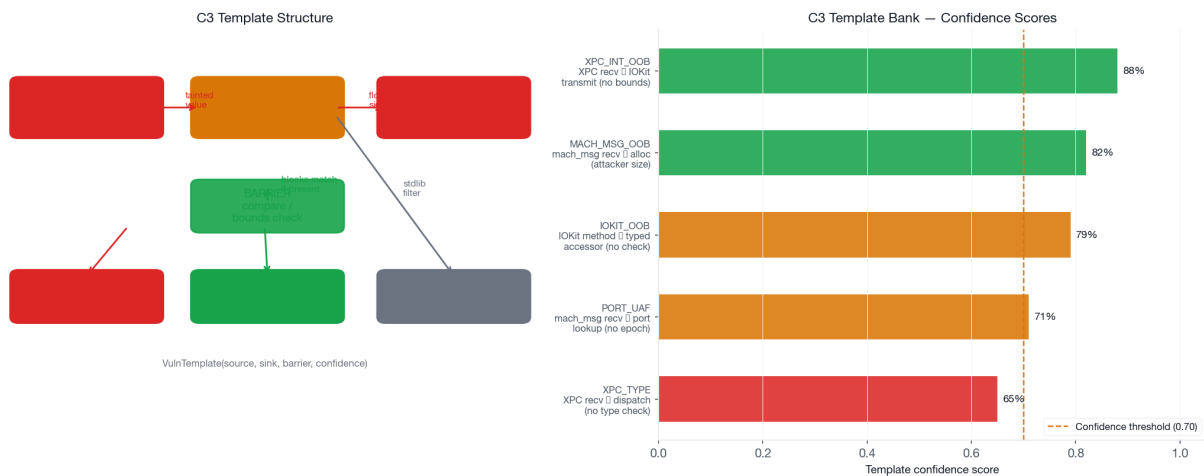


Figure 3 — C3 template structure (left): source → intermediate → sink pattern with barrier detection. Confidence scores for the five macOS vulnerability templates (right).

5.3 The five macOS templates

Template	Source	Sink	Barrier
XPC_INT_OOB	xpc_dictionary_get_int64()	IOKit copyScalar / alloc	bounds check compare
MACH_MSG_O...	mach_msg recv fields	malloc / alloc with size	size clamp
IOKIT_OOB	IOKit method argument	typed accessor	range check
PORT_UAF	mach_msg recv port field	port lookup table	epoch/generation check
XPC_TYPE	xpc_dictionary_get_value()	type-specific dispatch	xpc_get_type() call

§6

C6 — Symbolic Taint Analysis

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

6.1 Architecture: SimProcedure hooks

C6 hooks 18 security-relevant functions via angr SimProcedures. When the symbolic engine calls a hooked function, the hook intercepts the call and returns a fresh claripy BVS (bitvector symbol) tagged with a taint label. This creates a "taint source" in the symbolic state that flows through the program via normal constraint propagation — no separate taint tracking required.

```
def Hook_malloc(state):
    size = state.solver.eval(state.regs.x0)    # ARM64: size in x0
    taint = claripy.BVS(f"malloc_ret_{self.call_id}", 64)
    taint_map[taint.args[0]] = {"type": "malloc", "size": size}
    return taint
```

6.2 OOB, UAF, and type confusion detection

At sink functions, C6 checks whether symbolic values carry taint labels and whether the constraints on those values admit a bug witness. OOB detection checks whether the solver can find an assignment where an index exceeds its allocation size. UAF detection checks whether a dereferenced pointer shares a taint label with a freed allocation. Type confusion detection checks whether the symbolic type tag at an accessor differs from the expected type for that call site.

Solver interaction — confidence from max()

For each taint source, C6 asks the solver: $\max(\text{index}) \geq \text{alloc_size}$

If yes AND the concrete max value is solver-confirmed: ALERT with concrete PoC input.

If yes but solver times out: LOW CONFIDENCE alert (symbolic only).

`solver.max()` is used rather than `solver.satisfiable()` to get a PoC value directly.

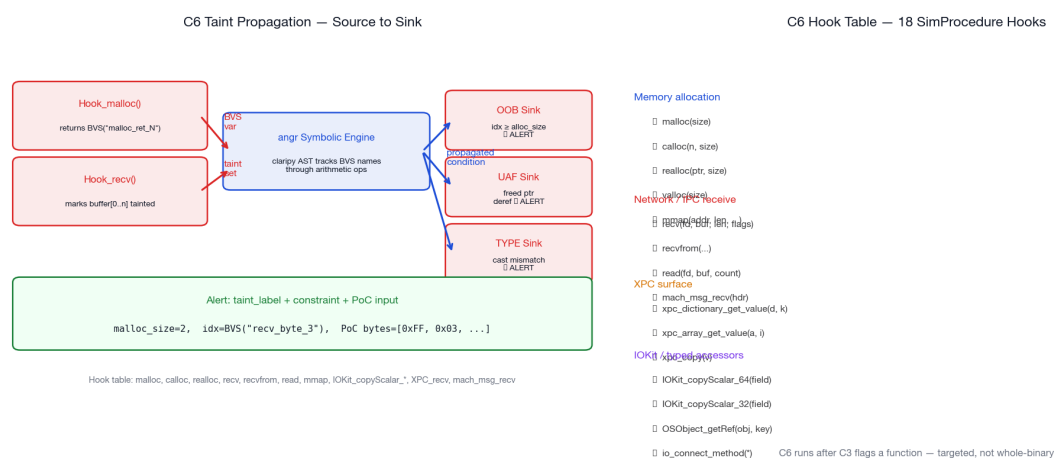


Figure 4 — C6 taint propagation: 18 source hooks tag return values as BVS; angr propagates taint through arithmetic; sink hooks detect OOB/UAF/type confusion.

6.3 The hook table

Hook(s)	Category	Taint target
malloc, calloc, realloc, valloc, mmap	Memory allocation sources	size argument + return value
recv, recvfrom, read	Network / file input sources	buffer + count
mach_msg_recv	Mach IPC receive	msg header fields

§7

The Disassembly Layer

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

7.1 VEX IR fundamentals

VEX IR (Valgrind IR) is a machine-independent intermediate representation. angr lifts each machine instruction to VEX IR statements. An IRSB (Intermediate Representation SuperBlock) corresponds approximately to one basic block. Key statement types:

- WrTmp(t, expr) — write expr to temporary tN (SSA within the block)
- Put(offset, expr) — write to guest register file (x0–x28, sp, pc)
- Store(addr, expr) — write to memory
- IMark(addr, len) — instruction boundary marker (not executable)
- Exit(guard, target) — conditional branch (creates CFG edge)

Critical limitation: SSA scope is per-IRSB only

VEX temporaries are SSA only within a single IRSB. Across basic block boundaries, register state is tracked via Put/Get (register file) and Store/Load (memory). C3 must reconstruct cross-block dataflow manually using the canonical address scheme described in §5.2. Attempting to read cross-block VEX temporaries is a common error that produces incorrect (empty) taint sets.

7.2 Compiler constant-folding — the VEX IR trap

When scanning VEX IR for arithmetic patterns (e.g., "oip + 0x14" for an IP header offset bug), the expected Binop(lOp_Add64, RdTmp(t_oip), Const(0x14)) statement often does not appear. ARM64 compilers constant-fold pointer arithmetic at compile time: instead of a runtime add, the load instruction encodes the offset directly as an immediate. The correct fix is to scan load/store offsets in otool disassembly output rather than VEX Add64 intermediate values.

```
# WRONG: scanning VEX for Add64(RdTmp, Const(0x14)) finds nothing
# CORRECT: otool sliding-window scan for consecutive:
#   LDRB  w_N, [xB, #0x1c]  ← ip->ip_hl byte (offset fixed)
#   LDRH  w_M, [xB, #0x20]  ← inner IP field (same base reg xB)
# If both share the same base register → fixed-offset logic bug confirmed
```

7.3 Windows PE: pefile + capstone

angr's CFGFast does not reliably resolve Windows IAT thunk names in a single-session analysis — import table entries are runtime-resolved addresses, not static call targets. The fix: use pefile.PE() to build an IAT map (import.address → name) and resolve call targets by computing the RIP-relative IAT slot from capstone disassembly:

```
pe = pefile.PE("target.exe")
iat = {imp.address: imp.name.decode()
        for entry in pe.DIRECTORY_ENTRY_IMPORT
        for imp in entry.imports if imp.name}

# Resolve CALL [rip + disp]:
# slot_va = instr.address + instr.size + disp
# function_name = iat.get(slot_va, f"sub_{slot_va:x}")
```

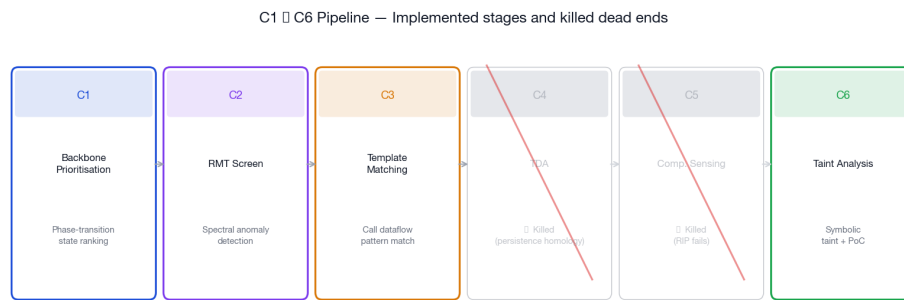
7.4 Binary acquisition without a VM

§8

Pipeline Composition

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

8.1 End-to-end flow



C4 and C5 killed by LLM purple-team architecture review — gaps preserved, not renumbered

Figure 6 — Full pipeline from binary collection through PoC. C4 and C5 are struck out — killed by architecture review.

8.2 Stage interface contracts

Stage	Input	Output	Persisted format
C1	angr.SimState	angr SimState + backbo...	Priority stash for simgr
C2	Mach-O / PE binary	C2RMTRResult + ranked f...	JSON: top_addrs.json
C3	top_addrs.json + binary	C3HitList	JSON: c3_hits.json
C6	c3_hits.json + binary	C6Alert list	JSON: c6_alerts.json + PoC

8.3 Dependency and fallback behaviour

- C1 requires an active angr.SimulationManager — it is a SimulationManager technique, not a standalone script. It runs transparently inside any angr symbolic execution session.
- C2 operates on CFGFast output. If CFGFast fails (arm64e PAC, chained fixups), C2 falls back to otool-based function enumeration with McCabe only (no spectral metrics).
- C3 reads C2 top_addrs.json by default. If not present, it runs C2 first. Can be invoked standalone with an address list.
- C6 requires angr symbolic execution to be viable on the target function. If angr exceeds memory/time limits, C6 is skipped and C3 results stand.

angr arch override — always required on macOS

Universal Mach-O contains x86_64 and arm64e slices. angr defaults to x86_64 even on Apple Silicon. Always pass explicit arch:

```
proj = angr.Project(path, auto_load_libs=False,
                    main_opts={"arch": archinfo.arch_from_id("aarch64")})
```

Do NOT pass arch as a bare string — angr requires an archinfo.Arch instance.

§9

Limitations & Dead Ends

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

9.1 Known tool limitations

Limitation	Root cause	Workaround
angr CFG size limit	~3.3 MB Mach-O	Sparse eigenvalue approx for $N > 2000$; skip if OOM
arm64e PAC	CLE chained fixups	Falls back to otool symbol scan
ObjC dispatch / vtable	Indirect calls	CFGFast misses; manual inspection only
C++ STL / Swift stdlib	False positives	Filter by callee name prefix in C3/C6
Windows ARM64 CC	Missing Win32 entry	Monkey-patch SYSCALL_CC before Project()
Compiler folding	VEX Add64 absent	Use otool LDRB/LDRH sliding-window scan

9.2 Why C4 (TDA) was killed

Topological data analysis on CFGs computes the persistent homology of the basic block graph — the "shape" of the control flow over different scales of connectivity. This is mathematically elegant. It is also operationally useless for this pipeline, because loop structure and branching depth — the features TDA would detect — are already captured by back-edge count (loops) and cyclomatic complexity (branching depth), both of which C2 computes in microseconds from simple graph traversal. TDA would take seconds per function via gudhi or ripser and add no discriminative signal beyond what we already have. The kill decision was unanimous across all four LLMs in the architecture review.

9.3 Why C5 (compressed sensing) was killed

The proposal was to treat AFL++ coverage bitmaps as sparse signals and apply compressed sensing — projecting the 64K-entry bitmap down to a small number of random linear measurements, then recovering the original from those measurements. This requires the measurement matrix Φ to satisfy the Restricted Isometry Property (RIP): for all k -sparse vectors x , $\|\Phi x\|_2 \approx \|x\|_2$. The RIP holds for random Gaussian/Bernoulli measurement matrices — but coverage bitmaps are binary vectors with highly structured sparsity. Most branches are taken the same way across most inputs: the sparsity pattern is correlated, not random. The RIP fails, random projection cannot recover the original, and the compressed representation carries less information than a simple hash. Killed.

General principle

Mathematical elegance is not the same as operational utility.

Both TDA and compressed sensing are legitimate techniques in their domains.

They fail here because the assumptions they require do not hold for call graphs and coverage bitmaps. Document failure modes — they inform the next design cycle.

§10

Glossary

Mathematics, Physics & Implementation — C1–C6 Pipeline Reference

Terms and notation

3-SAT	Satisfiability problem with 3-literal clauses. Phase transition at $\alpha_c \approx 4.267$.
α_c	Critical clause/variable ratio for 3-SAT phase transition (≈ 4.267).
α_{cond}	Condensation transition ratio ($\approx 4.15\text{--}4.27$), where solutions cluster.
Backbone	Set of variables taking the same value in every satisfying assignment.
BVS	Bitvector symbol — claripy's symbolic variable type. Carries taint via name.
C1–C6	Pipeline stage designators. C4, C5 killed by design review; gaps preserved.
CDCL	Conflict-Driven Clause Learning — the algorithm inside Z3 and most SAT solvers.
CFG	Control Flow Graph. Nodes = basic blocks, edges = branches.
CFGFast	angr's fast CFG recovery (linear sweep + recursive descent, no execution).
Configuration model	Correct RMT null for call graphs: random rewiring preserving degree sequence.
Cyclomatic complexity	$M = E - N + 2$. Number of independent paths through a function's CFG.
Eigenvector centrality	Importance of a node weighted by importance of its neighbours (power iteration).
Graph energy	$E(G) = \sum \lambda_i / N$. Sum of absolute eigenvalues per node.
IAT	Import Address Table — Windows PE runtime-resolved function pointers.
IRSB	Intermediate Representation SuperBlock — one basic block in VEX IR.
Marchenko-Pastur	RMT null distribution for i.i.d. random matrices. Wrong for call graphs.
OOB	Out-Of-Bounds read or write past an allocated buffer boundary.
PAC	Pointer Authentication Code — ARM64e hardware pointer signing.
Power iteration	Algorithm for computing the principal eigenvector: $x^{(k+1)} = Ax^{(k)} / \ Ax^{(k)}\ $.
QF_BV	Quantifier-Free Bitvector Theory — angr's constraint language.
RIP	Restricted Isometry Property — required for compressed sensing recovery.
RMT	Random Matrix Theory — study of eigenvalue spectra of large random matrices.
SimProcedure	angr hook that replaces a function's body with a Python implementation.
Spectral radius	$\rho(A) = \max \lambda_i $. Largest eigenvalue magnitude.
TDA	Topological Data Analysis — persistent homology. Killed at C4.
Tseitin transform	Converts arbitrary boolean formulae to CNF. Destroys factor-graph structure.

UAF	Use-After-Free — dereferencing a pointer after the allocation was freed.
VEX IR	Valgrind IR — machine-independent intermediate representation used by angr.
Wigner semicircle	RMT null for symmetric i.i.d. random matrices. Wrong for call graphs.
WrTmp	VEX IR statement: assign expression to SSA temporary tN.
z-score	$z = (\text{obs} - \mu) / \sigma$. Measures deviation from null distribution in sigma units.

Document Information

Generated by generate_toolchain_docs.py · 2026-04-17

Stuart Thomas · Independent Security Researcher

Findings referenced are redacted — this document covers methodology only.

Repository: [macos_vuln_toolchain/angr_hardness/](#)